Day 152: Recurrent Networks for Forecasting: Week 10's LSTM against ARIMA, Regression and a Control

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 22, Day 6 | Complete study guide

Watch: Long Short-Term Memory with PyTorch + Lightning (StatQuest with Josh Starmer) | Multi-Step LSTM Time Series Forecasting (TravelWithDatascientist)

Script: `day152_lstm_forecasting.py` · every number in this guide is copied by program from `day152_run_output.txt`, the real output of that script.

Week 10's LSTM, imported unchanged from Day 65's exercise, reads the last 24 months and predicts the next; further months come by feeding each prediction back in. Trained on CO2's levels, the networks never forecast above the highest training month (370.4 against 371.0). Trained on month-to-month changes, five networks averaged forecast 2000–2001 with an error of 0.250 ppm, against the airline ARIMA's 0.412. But a control with no network at all, each calendar month's average change added up, scored 0.231. On rolling origins the LSTM beat that control, and a version of the control using the last 10 years beat the LSTM at most horizons.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

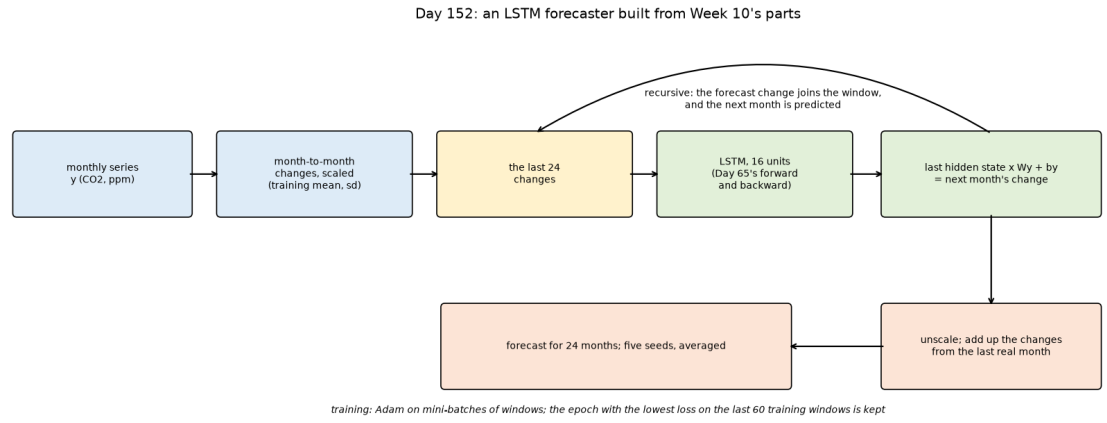
A **recurrent network** reads a sequence one step at a time and carries a memory (its hidden state) from step to step. Week 10's **LSTM** adds gates that decide what the memory keeps, adds and shows. Used as a forecaster, it reads a **window** of the last 24 months, and a linear layer turns its final hidden state into one number: next month.

To forecast further, the prediction is appended to the window and the network runs again: the same **recursive** strategy as Day 151's regression. Training is by gradient descent on many windows from the training years.

REAL-WORLD SCENARIO

A sales team asks whether a neural network would forecast demand better than the spreadsheet method it uses now (last year plus the usual seasonal growth). The honest answer needs both in the same test, on the same months, and the spreadsheet method may well win.

SYSTEM DIAGRAM



The forecaster: changes, a window, Day 65's LSTM, one output, fed back in.

Today's part for the week's build

`day152_exercise.py` is today's parts (windows, the network's weights, its loss and gradients through Day 65's LSTM, prediction, Adam training with the best validation epoch kept, the recursion, the fit on changes, the forecasts, and the no-network control), checked by hand: the windows of 1..6, the starting biases, the loss and one gradient of an all-zero network, a numerical gradient check, training on a constant series, the recursion and the adding-up for constant networks, and the control on a steadily rising series.

ACTUAL OUTPUT

```
windows of 1..6, length 3: inputs [1,2,3], [2,3,4], [3,4,5], targets 4, 5, 6; gives [[1.0, 2.0,
    3.0], [2.0, 3.0, 4.0], [3.0, 4.0, 5.0]], [4.0, 5.0, 6.0]
starting biases, 2 hidden units: forget gates 1, the rest 0; gives [1.0, 1.0, 0.0, 0.0, 0.0, 0.0,
    0.0, 0.0]
all-zero network, targets 1 and 3: output 0, loss (1 + 9) / 2 = 5, gradient for by = 2 x mean(0 -
    target) = -4; gives 5.0, -4.0
numerical gradient check, every weight of a 3-unit network: largest |numeric - analytic| 2.6e-11
trained on a series that is always 0.5: predicts 0.499 (validation loss 3.5e-07 at epoch 40)
recursion with a network that always outputs 0.7: 0.7, 0.7, 0.7; gives [0.7, 0.7, 0.7]
changes of 2 added up from 16 (history 10, 12, 14, 16): 18, 20, 22; lstm_forecast gives [18.0,
    20.0, 22.0]
control on 0, 1, ..., 24 (every change 1): 25, 26, 27; seasonal_drift gives [25.0, 26.0, 27.0]
```

Data: Day 147's two series. CO2 is trained up to 1999-12 and tested on 2000–2001, as on Days 147–151; the orders are used on a log scale. The window (24 months), the network size (16 units), the learning rate and the 300 epochs were fixed in advance, not tuned; the epoch kept for each network is the one with the lowest loss on the last 60 training windows.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. The forecaster, and a gradient check through all of it

DEFINITION — start here, no prior knowledge assumed

Day 65's exercise gave the LSTM's forward pass over a sequence and the backward pass for a gradient that arrives at the last hidden state. Today adds a linear output (16 weights and a bias), a mean squared error, and **Adam**, which scales every step by a running average of the gradient's size. The check nudges every one of the network's weights and compares the change in loss with the computed gradient.

REAL-WORLD SCENARIO

Checking a bank statement by hand for a few accounts before trusting the bank's software for all of them.

MATHEMATICS

prediction: $y\text{-hat} = h_{24} \cdot W_y + b_y$, h_{24} = *the LSTM's hidden state after the window*

loss $= \text{mean}\,(y\text{-hat} - y)^2$; $d\text{loss} / d\,y\text{-hat} = 2\,(y\text{-hat} - y) / m \;\rightarrow$ *into Day 65's backward pass at* h_{24}

Adam: $m \leftarrow 0.9\,m + 0.1\,g$, $v \leftarrow 0.999\,v + 0.001\,g^2$, $w \leftarrow w - 0.01\,m\text{-hat} / (\sqrt{v\text{-hat}} + 10^{-8})$

IMPLEMENTATION

```
def loss_and_grads(P, X, Y):
    m, H = len(X), P["Wy"].shape[0]
    z0 = np.zeros((m, H))
    Hs, Cs, gates = d65.lstm_forward(X, z0, z0, P["W"], P["b"])
    hT = Hs[:, -1]
    r = hT @ P["Wy"] + P["by"] - Y
    dy = 2 * r / m
    dW, db = d65.lstm_backward(np.outer(dy, P["Wy"]), X, z0, z0, P["W"], Hs, Cs, gates)
    return np.mean(r ** 2), dict(W=dW, b=db, Wy=hT.T @ dy, by=np.array([dy.sum()]))
```

IMPLEMENTATION

```
def adam_train(X, Y, Xv, Yv, H, seed, epochs=EPOCHS, lr=0.01, bs=32):
    P = init_params(H, seed); m1 = {k: 0 * v for k, v in P.items()}; m2 = {k: 0 * v for k, v in
        P.items()}
    r, k = np.random.RandomState(seed + 100), 0
    best, hist = (np.inf, None, 0), []
    for ep in range(1, epochs + 1):
        idx = r.permutation(len(X)); tl = 0.0
        for s in range(0, len(X), bs):
            j = idx[s:s + bs]; loss, g = loss_and_grads(P, X[j], Y[j]); k += 1; tl += loss *
                len(j)
            for q in P:
                m1[q] = 0.9 * m1[q] + 0.1 * g[q]; m2[q] = 0.999 * m2[q] + 0.001 * g[q] ** 2
                P[q] -= lr * (m1[q] / (1 - 0.9 ** k)) / (np.sqrt(m2[q] / (1 - 0.999 ** k)) + 1e-8)
        vl = np.mean((predict(P, Xv) - Yv) ** 2); hist.append((tl / len(X), vl))
        if vl < best[0]:
            best = (vl, {q: w.copy() for q, w in P.items()}, ep)
    return best[1], best[2], np.array(hist)
```

ACTUAL OUTPUT

```
nudge 1e-06: worst relative error 3.3e-04 (absolute 2.3e-10, on a weight whose gradient is 3.5e-07)
nudge 1e-05: worst relative error 1.3e-05 (absolute 9.3e-12, on a weight whose gradient is 3.5e-07)
first draft used only the 1e-06 nudge and saw 3.3e-04: the worst weight's gradient is tiny, and rounding in the
two losses swamps the difference; the larger nudge shows the gradients are right
network: a window of 24 months -> LSTM with 16 hidden units -> one number; 1169 weights
Actual output from this lesson's run.
```

What it means

With nudges of 1e-05 the worst relative error is 1.3e-05, with an absolute error of 9.3e-12 on that weight: the gradients are right.

A first-draft scare, kept on purpose: with nudges of 1e-06 the worst relative error was 3.3e-04, too large for a correct gradient. It sat on a weight whose gradient is only 3.5e-07; with such a small change in the loss, rounding in the two loss values swamps the difference, and a larger nudge removes the problem. The network has 1169 weights.

WHY CHOOSE THIS?

The same network trains on any window of any series, with no model of trend or season written in.

WHY NOT THIS? (TRADEOFFS)

Many more weights than ARIMA's two coefficients, a training run instead of a solve, and results that depend on the seed.

2. Levels against changes

DEFINITION — start here, no prior knowledge assumed

Five networks were trained on CO2's scaled **levels**, and five on its month-to-month **changes**. A forecast from changes adds each predicted change to the last real level, so it can rise past anything seen in training. A forecast of levels must itself produce the new, higher values.

REAL-WORLD SCENARIO

Predicting a child's height directly from past heights, or predicting how much they will grow this year and adding it on: only the second extends naturally past the tallest height in the records.

IMPLEMENTATION

```
def fit_lstm(y, L=L, H=HID, seeds=SEEDS, epochs=EPOCHS, nval=NVAL, diff=True):
    d = np.diff(y) if diff else y
    mu, sd = d.mean(), d.std(); z = (d - mu) / sd
    X, Y = windows(z, L)
    fits = [adam_train(X[:-nval], Y[:-nval], X[-nval:], Y[-nval:], H, s, epochs) for s in seeds]
    return dict(models=[f[0] for f in fits], epochs=[f[1] for f in fits], hist=[f[2] for f in
        fits], mu=mu, sd=sd, L=L, diff=diff)
def lstm_forecast(F, y, h):
    d = np.diff(y) if F["diff"] else y
    z = (d - F["mu"]) / F["sd"]
    out = []
    for P in F["models"]:
        f = recursive(P, z, F["L"], h) * F["sd"] + F["mu"]
        out.append(y[-1] + np.cumsum(f) if F["diff"] else f)
    return np.array(out)
```

ACTUAL OUTPUT

```
10 networks trained (149.3 s)
trained on levels: test MAE by seed 4.83, 4.43, 3.78, 2.00, 2.30 ppm
  the training months reached 371.0 ppm and the test months reach 373.8; the networks' forecasts reach at most 370.4, below
      the training maximum
  the hard bound (|h| < 1, so output < sum|Wy| + by) is far higher: 406.1, 411.2, 421.9, 409.3, 415.9 -- it is not what held
      them back
trained on month-to-month changes: test MAE by seed 0.224, 0.279, 0.259, 0.390, 0.313 ppm
a forecast change is added to the last real level, so the forecast can rise past anything in training
Actual output from this lesson's run.
```

What it means

Trained on levels, the networks forecast 2000–2001 with errors of 2.00 to 4.83 ppm, and no forecast went above 370.4, just below the highest training month (371.0), while the test months reach 373.8. The output does have a hard limit, since every hidden unit lies between –1 and 1, but here it is far higher (406.1 to 421.9), so it is not what held them back. The networks simply never learned to produce values above those they were trained on; why, beyond that, was not tested here.

Trained on changes, the same networks scored 0.224 to 0.390 ppm.

WHY CHOOSE THIS?

Changes let a bounded network follow an unbounded rise, as differencing did for ARIMA on Day 148.

WHY NOT THIS? (TRADEOFFS)

Errors in the changes add up over the forecast: one month's mistake shifts every later month.

3. Forecasting CO2 in 2000–2001, with a control

DEFINITION — start here, no prior knowledge assumed

The five networks on changes forecast 24 months from 1999-12; their average is the method's forecast. They are compared with Day 149's airline ARIMA, Day 151's regression on lags 1–13, same month last year, and a **control**: no network, just the average change into each calendar month over the training years, added up from the last real month. If the network cannot beat that, it has learned nothing the averages did not already hold.

REAL-WORLD SCENARIO

Before crediting a new fertiliser, compare it with a plot that got nothing; before crediting a network, compare it with the simplest forecast built from the same information.

MATHEMATICS

$$control: \quad a_k = \text{average of } y_t - y_{t-1} \text{ over training months } t \text{ in calendar month } k$$

$$y\text{-}hat_{T+h} = y_T + a_{month(T+1)} + \ldots + a_{month(T+h)}$$

IMPLEMENTATION

```
def seasonal_drift(y, start, h, years=None):
    d = np.diff(y)
    mo = (np.datetime64(start, "M").astype(int) + np.arange(1, len(y))) % 12
    if years:
        d, mo = d[-12 * years:], mo[-12 * years:]
    avg = np.array([d[mo == k].mean() for k in range(12)])
    fm = (np.datetime64(start, "M").astype(int) + len(y) + np.arange(h)) % 12
    return y[-1] + np.cumsum(avg[fm])
```

ACTUAL OUTPUT

```
best epoch on validation, by seed: [274, 217, 158, 139, 285] of 300; validation loss at the best epoch 0.0594, 0.0675,
    0.0597, 0.0645, 0.0606 (scaled changes; 1 = their variance)
training loss at the last epoch 0.0456, 0.0552, 0.0474, 0.0463, 0.0421: below validation, as expected
2000-01 to 2001-12, MAE (ppm): LSTM seeds 0.224, 0.279, 0.259, 0.390, 0.313 (range 0.224 to 0.390); average of the five
    0.250
  airline ARIMA 0.412; regression on lags 1-13 (Day 151) 0.706; same month last year 1.882
the seed alone moves the error by 0.165 ppm; one network's score is not the method's score
the control, no network (each calendar month's average change over all training years, added up): 0.231
  the same control from the last 10 years 0.275, the last 5 0.419: the window matters, and picking it now would be choosing
      on the test
bias (forecast - actual, mean): LSTM average +0.196, airline ARIMA +0.405, control +0.066
Actual output from this lesson's run.
```

What it means

The best validation epochs were [274, 217, 158, 139, 285], and the training loss stayed below the validation loss, as it should. The seed alone moved the test error from 0.224 to 0.390, by 0.165 ppm: one network's score is not the method's score. The average of the five scored 0.250, better than the airline ARIMA (0.412) and far better than the regression (0.706).

The control scored 0.231, better than the networks. The two shorter windows (0.275 for 10 years, 0.419 for 5) were tried after the all-years result, only to see whether the window matters; it does, and choosing one now would be choosing on the test. Much of the airline model's error is bias: it forecast too high by +0.405 on

average, the LSTM by +0.196, the control by +0.066.

WHY CHOOSE THIS?

Averaging several seeds gives a steadier forecast than any one network, at no extra modelling cost.

WHY NOT THIS? (TRADEOFFS)

On 24 test months, a network with 1169 weights did not beat twelve averages; a control showed it.

4. Rolling origins, every model fitted once

DEFINITION — start here, no prior knowledge assumed

Retraining five networks at each of 97 origins would be slow, so every model is fitted once on the months up to 1989-12 and then forecasts 24 months from each origin of 1990–1998, reading the real history up to that origin. The airline model keeps its 1989 coefficients, the regression its 1989 weights, and the control its 1989 averages, so the comparison is like for like.

REAL-WORLD SCENARIO

Judging forecasters who all trained on the same old data and are then asked about many later months: a fair test of what each learned.

ACTUAL OUTPUT

```
97 origins (67.7 s, including training)
   MAE (ppm), months ahead       1       6      12      24
        LSTM, average of 5   0.252   0.534   0.630   0.949
        LSTM, single seeds   0.263   0.543   0.648   0.980
             airline ARIMA   0.264   0.453   0.717   1.101
     regression, lags 1-13   0.314   0.650   0.736   1.127
      same month last year   1.362   1.428   1.560   3.137
        control, all years   0.278   0.641   0.680   1.077
    control, last 10 years   0.251   0.452   0.637   0.908
the averaged LSTM beats the airline model at 18 of 24 horizons; averaging the seeds beats the typical single seed at 24 of
    24
against the control: the averaged LSTM is better than all years' averages at 24 of 24 horizons, and than the last 10 years'
    at 4 of 24
Actual output from this lesson's run.
```

What it means

The averaged LSTM beat the airline model at 18 of 24 horizons (0.949 against 1.101 at 24 months), and averaging the seeds beat the typical single seed at every horizon. It beat the all-years control everywhere (1.077 at 24), so over these years the networks did learn something the long-run averages did not hold.

The control built from the last 10 years before 1990 did better still at most horizons: the LSTM won at only 4 of 24 (0.908 against 0.949 at 24 months). A likely reason, not tested here: CO2's yearly rise grew over the decades, so recent changes describe the 1990s better than the average since 1958.

WHY CHOOSE THIS?

Fitted once, the network forecast well for nine years without retraining.

WHY NOT THIS? (TRADEOFFS)

A simple average over the right window did as well; which window is right is itself a choice to make on past data only.

5. Electrical-equipment orders

DEFINITION — start here, no prior knowledge assumed

The same forecaster on the log orders, trained up to 2007-12 (the crash) and up to 2014-05 (the last two years). With fewer months here the validation block is the last 24 windows.

REAL-WORLD SCENARIO

A demand model trained on a few years of a volatile market: the question is how much its answer depends on luck.

ACTUAL OUTPUT

```
the crash: train up to 2007-12 (131 windows, the last 24 for validation); 24 months, MAE:
  LSTM seeds 22.58%, 28.01%, 24.29%, 22.25%, 32.42%; average 25.57%; airline ARIMA 23.86%; same month last year 18.04%;
      control 23.06%
the last 24 months: train up to 2014-05 (208 windows, the last 24 for validation); 24 months, MAE:
  LSTM seeds 2.19%, 3.45%, 2.48%, 10.47%, 12.82%; average 5.30%; airline ARIMA 6.41%; same month last year 3.75%; control
      2.37%
Actual output from this lesson's run.
```

What it means

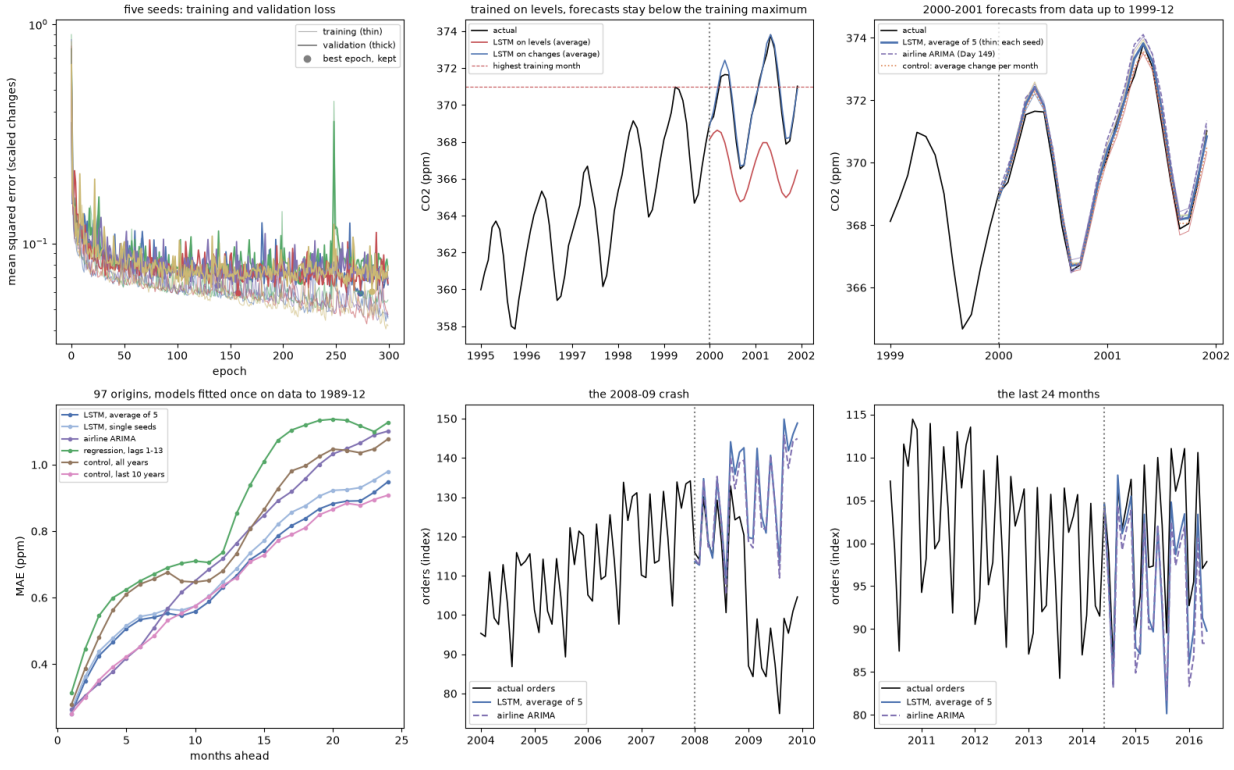
Through the crash, the networks averaged 25.57%, worse than the airline model (23.86%), the control (23.06%) and same month last year (18.04%). Over the last two years the seeds ranged from 2.19% to 12.82%: with this little data, the seed decides more than the method. The control was best (2.37%).

WHY CHOOSE THIS?

Nothing in the network is specific to CO2; the same code ran on the orders unchanged.

WHY NOT THIS? (TRADEOFFS)

With a few hundred windows, five seeds disagreed by a factor of nearly six; small series favour small models.



Top: training and validation loss for the five seeds (dots: the epoch kept); networks trained on levels against changes; the 2000–2001 forecasts with the control. Bottom: rolling-origin error by horizon (same month last year is off the scale; its numbers are printed); the orders through the crash and over the last two years.

Interview preparation — technical questions

Q: How do you set up an LSTM to forecast a series?

A: Windows of past values as inputs, the next value as target; a linear layer on the last hidden state; recursion for longer horizons.

Q: Why train on differences rather than levels?

A: Here networks trained on levels never forecast above 370.4 ppm, below the training maximum of 371.0, while the test reached 373.8; on changes the error fell to 0.224-0.390.

Q: How do you know a network beats a simple method?

A: Put a control in the same test: each month's average change, added up, scored 0.231 against the networks' 0.250.

Q: Why average several seeds?

A: Seeds moved the CO2 test error by 0.165 ppm and the orders' by a factor of nearly six; the average beat the typical single seed at all 24 horizons.

Q: How do you choose the training epoch without the test?

A: Keep the epoch with the lowest loss on a validation block at the end of the training period.

Interview preparation — theoretical questions

Q: What does the LSTM's memory add over a regression on lags?

A: Its hidden state can summarise the whole window non-linearly; on rolling origins it beat lags 1-13 at every horizon shown.

Q: Why can a gradient check fail on correct code?

A: When a weight's gradient is tiny, rounding in the two losses swamps a small nudge: 3.3e-04 at 1e-06, 1.3e-05 at 1e-05.

Q: Why fit once and roll?

A: Retraining at every origin is costly for networks; fitting every model once keeps the comparison fair.

Q: When do statistical models tend to beat networks?

A: On short single series: here the orders gave a few hundred windows, and the control and last year's value won.

Key takeaway and what's next

KEY TAKEAWAY

Week 10's LSTM became a forecaster with a linear output, a recursion and a choice of target. On levels it never forecast above its training range; on changes it beat the airline ARIMA, both on 2000–2001 and over 97 rolling origins. A control with no network, each month's average change added up, was the sobering part: it won on the test months, and its 10-year version matched or beat the network on rolling origins. On the short, volatile orders series, the seed mattered more than the method. Tomorrow's build adds this week's ideas up into one decomposable forecaster.

- Gradients checked: 1.3e-05 relative with 1e-05 nudges (first draft: 3.3e-04 at 1e-06, rounding).
- Levels: forecasts never above 370.4 ppm; changes: 0.224–0.390.

- CO2 2000–2001: LSTM average 0.250, airline 0.412, control 0.231.
- Rolling: LSTM beat the airline model at 18 of 24 horizons; the 10-year control beat the LSTM at most.
- Orders, last two years: seeds 2.19% to 12.82%; control 2.37%.

Suggested watch order, if starting from zero

- 1. Long Short-Term Memory with PyTorch + Lightning (StatQuest with Josh Starmer) — an LSTM unit coded from scratch and trained, to set beside Day 65's parts used today.
- 2. Multi-Step LSTM Time Series Forecasting (TravelWithDatascientist) — windowed LSTM forecasting several steps ahead, the setting of today's experiments.

Day 153 preview

- **The Week 22 build: a Prophet-style forecaster.** A trend that may bend at changepoints, a season made of Fourier waves (Day 151), fitted by least squares, assembled from this week's parts and compared with every model of the week.